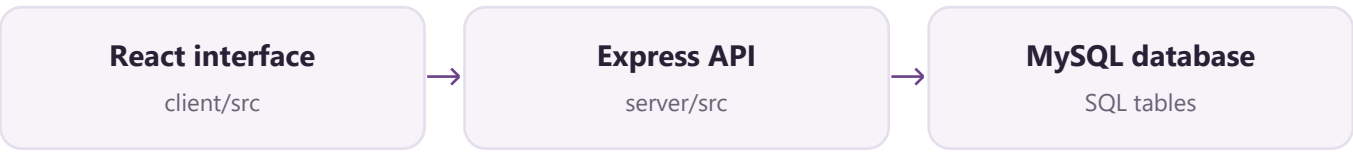


Project structure

University Lab & Equipment Booking System

The system lets students and faculty request university resources, lab staff handle approvals and equipment returns, coordinators manage department rules, and administrators oversee users and system activity. Its code is split into three main parts.



Important folders in the main directory

Folder / file	Purpose
client/	React web application. Contains screens, navigation, reusable interface components, styles, and branding.
server/	Node.js / Express API. Handles authentication, permissions, bookings, resource management, notifications, and database access.
shared/	Role definitions and capability rules shared by the frontend and backend.
server/tests/	Workflow and role-access regression scripts: workflow.cjs and roleMatrix.cjs .
output/	Supporting deliverables and previous verification artifacts; not application source.
node_modules/ client/dist/	Generated dependencies and the compiled frontend respectively. Source changes belong in client/src or server/src.

Important root files

package.json defines the client/server workspaces and commands to run, build, start, and seed the project.

package-lock.json records dependency versions. **.gitignore** excludes selected local and generated files.

README.md introduces the project. **PROJECT_EXPLANATION.md**, **REQUIREMENTS_COVERAGE.md**, **ROLE_ACCESS_MATRIX.md**, and **VERIFICATION.md** provide supporting design, requirement, permission, and verification notes.

Current implementation: React 19 + Vite 6, Express 5, and MySQL. The root docker-compose.yml provisions MongoDB and is a legacy configuration; the current database connection is defined in server/src/config/db.js.

The user interface

Paths below are relative to `client/`. The frontend presents the workflow for the signed-in user and communicates with the API.

Folder / file	Purpose
index.html vite.config.js	Browser HTML entry and Vite development/build configuration.
src/main.jsx	Mounts the React application and loads the global stylesheet.
src/App.jsx	Connects the router, authentication provider, public pages, and protected pages. Displays the launch splash for 2.5 seconds.
src/pages/	Complete screens: home and authentication; dashboard and resources; booking requests/history; issue and return operations; management, rules, analytics, users, notifications, and activity.
src/layouts/ AppLayout.jsx	Shared authenticated workspace shell, including sidebar navigation and the page content area.
src/context/ AuthContext.jsx	Stores the current user, performs login/register/logout, restores sessions, and exposes capability checks from the shared role policy.
src/api/http.js	Common HTTP client. Sends JSON requests, attaches the bearer token, and handles API responses and errors. Uses <code>VITE_API_URL</code> for the API address.
src/components/	Reusable UI pieces such as Modal, PageHeader, StatusPill, and StatCard. ProtectedRoute checks page access; SplashScreen and SystemBrand display branding.
src/assets/ hackathon.png	Original project logo asset used for the system branding.
src/styles/global.css	Shared typography, colors, spacing, responsive layouts, and component styling.
src/hooks/useAsync.js src/utils/format.js	Helpers for asynchronous screen data and consistent display formatting.

How the frontend connects

main.jsx → **App.jsx** → **AppLayout / pages** → **api/http.js**. A page collects input or requests data. The HTTP client sends it to the backend, and the page renders the response. AuthContext and ProtectedRoute use the shared policy to control navigation and access.

The server also enforces permissions on API requests; the interface alone is not the authorization boundary.

The API and data layer

Paths below are relative to server/src/ unless another prefix is shown.

Folder / file	Purpose
server.js / app.js	server.js connects MySQL, starts reminders, and listens for requests. app.js configures Express, security middleware, /api routes, and error handling.
routes/ / controllers/	Routes map endpoints to handlers; controllers translate requests into service calls and return responses. Domains include auth, resources, bookings, issues, analytics, notifications, users, and activity.
middleware/ validators/schemas.js	Authentication, capability enforcement, request validation, and consistent error responses. permissions.js reads the shared role policy.
services/	Business logic. bookingService handles the booking lifecycle; availabilityService checks resource/time conflicts; ruleService applies rules; recommendationService suggests resources or slots.
Other service files	issueService manages issue/return records. notificationService and activityService record updates; analyticsService summarizes usage; hydrationService assembles related data.
models/ db/repository.js	Models define access to users, departments, labs, equipment, bookings, rules, issue records, notifications, and activity. The repository converts operations into MySQL queries.
config/db.js config/schema.js config/env.js	MySQL connection pool, SQL table definitions, and environment configuration. The database schema is created on connection.
jobs/reminderJob.js utils/	Scheduled booking/return reminders and overdue updates; shared date, ID, error, async, and mutation helpers.
appwrite.js	Alternative Appwrite Function entry point that forwards requests to the same Express API and supports scheduled reminder execution.
server/scripts/seed.js	Creates initial demonstration resources and rules. This script can reset database tables.

One booking request, end to end

NewBookingPage → **api/http.js** → **/api/bookings** → **authentication, permissions and validation** → **booking controller** → **bookingService** → **models/repository** → **MySQL**. The service checks time, availability, and booking rules before saving. Notification and activity services record relevant updates; a JSON response returns to the page.

shared/rolePermissions.json connects both sides: student and faculty capabilities, lab staff operations, coordinator department duties, and administrator oversight. Frontend checks guide users; backend checks authorize actions.

Configuration values come from environment variables. Credentials are intentionally omitted from this document. This is a structure guide based on the current source, not a deployment or test-results report.